DATA MIND 3.0

A Verifier-Gated Architecture for Mathematical Search, Discovery, Learning, Installation, and Safe Upgrade

Brian Tenneson

Preliminary Expository Article – October 2026

Abstract

DATA MIND 3.0 is a modular architecture for difficult mathematical search in which intelligence, prediction, learned heuristics, speculation, resource allocation, and self-monitoring are deliberately separated from mathematical authority. Its governing principle is simple: no statement is permitted to become mathematically trusted merely because the component that produced it is intelligent, highly accurate, granted high clearance, or favored by the controller. Mathematical trust enters the system only through a Verifier. Verified results are retained in an append-only store called BANK, while speculative possibilities are kept separately in FUTUREBANK.[1]

Around this trust boundary, DATA MIND combines proof, refutation, independence, and consistency-oriented search; retrieval of relevant verified mathematics; alternative mathematical presentations; quotient and invariant discovery; heuristic direction-finding; bounded Scouts; resource allocation; partial-credit estimation; stagnation detection; criticism; creative proposal generation; revision; operational containment; long-term learning; and historical audit. The architecture is designed so that these mechanisms may become substantially more capable without silently acquiring the power to rewrite mathematical truth.

This article explains what DATA MIND is, what it can and cannot do, how a conforming implementation can be installed, how modules and learned components can be upgraded safely, how the system is used during an actual research run, and how it could approach several difficult mathematical situations. The central design goal is not merely stronger search. It is stronger search while preserving a visible distinction between conjecture, heuristic evidence, machine strategy, verified theorem, and operational safety.

1 What DATA MIND 3.0 Is

DATA MIND 3.0 is best understood as a mathematical operating architecture rather than as a single theorem prover or a single artificial-intelligence model.

It coordinates multiple specialized components, each with a restricted job and a defined interface. The most important distinction is between mathematical authority, mathematical strategy, and operational control. A Verifier decides whether a mathematical certificate is valid. Search modules propose candidate proofs, refutations, reductions, or consistency arguments. Heuristic modules estimate which directions are promising. A controller allocates resources and schedules modules. A safety layer governs permissions and containment. A learning layer improves routing and prediction. A history layer records what happened. None of these roles is silently interchangeable with another.

The architecture therefore treats a mathematical research system as a federation of bounded services rather than a single opaque agent. A useful implementation may contain powerful language models, theorem provers, symbolic engines, graph algorithms, learned rankers, retrieval systems, and specialized mathematical code. The design requirement is not that these components be weak. The requirement is that their authority remain explicit.

A compact view is

$$\text{Problem} \longrightarrow \text{Search} + \text{Retrieval} + \text{Representation} + \text{Heuristics} \longrightarrow \text{Candidate Certificates} \longrightarrow \text{Verifier} \longrightarrow \text{BAN}$$

In parallel,

$$\text{Speculative Proposals} \longrightarrow \text{FUTUREBANK},$$

and

$$\text{Operational Events} \longrightarrow \text{HISTORY}.$$

The controller may use information from all three regions, but it must not confuse them.

2 The Governing Trust Boundary

The most important architectural invariant is

$$\boxed{\text{Verifier} \neq \text{Controller} \neq \text{Heuristic Predictor}.}$$

This is not merely a software-engineering preference. It is the condition that allows the rest of the architecture to become more capable without making mathematical trust depend on model confidence.

Suppose a heuristic module assigns probability 0.9999 to a conjecture. That may justify allocating more search to the conjecture. It does not justify writing the conjecture into BANK.

Suppose a controller decides that a certain proof route is extremely promising. That may justify spawning Scouts or increasing a search budget. It does not make the route correct.

Suppose a high-clearance component produces a theorem statement. Clearance governs permission, not truth. The theorem still requires a certificate accepted by the Verifier.

This yields a simple rule:

$$\text{mathematical trust} \iff \text{verifier acceptance under the declared formal boundary}.$$

The architecture may also support empirical or scientific work, but empirical support must be labeled differently from formal proof. A numerical experiment, simulation, or dataset may provide strong evidence without becoming a formal theorem.

3 The Core Modules

A conforming DATA MIND 3.0 installation may contain many modules, but the following roles capture the intended architecture.[1]

3.1 Verifier

The Verifier is the sole mathematical authority. It checks proof objects, refutations, certificates, or other formally declared evidence. Its acceptance rules must be fixed for a run or versioned explicitly.

3.2 BANK

BANK is append-only trusted mathematical memory. Entries require verifier acceptance and retain provenance sufficient to reconstruct why they were accepted. A BANK entry should not merely say that a theorem is true; it should preserve the accepted certificate or a stable reference to it.

3.3 FUTUREBANK

FUTUREBANK stores speculative material: conjectures, candidate lemmas, heuristic patterns, incomplete arguments, possible analogies, and proposals that may later become useful. FUTUREBANK is intentionally not trusted memory.

3.4 Search Kernel

The Search Kernel manages theorem-proving activity. It may coordinate proof search, refutation search, model construction, consistency-oriented exploration, independence-oriented exploration, and bounded search over formal objects.

3.5 Relevant-Math Retriever

The Retriever locates verified results, definitions, known structures, and prior BANK entries that may reduce the current problem. Retrieval is advisory unless the retrieved item already carries an accepted certificate under the current trust boundary.

3.6 Alternative-Math Presenter

This module changes representation. It may rewrite a problem using graphs, algebraic objects, coordinates, quotient structures, invariants, normal forms, combinatorial encodings, or other mathematically equivalent presentations.

A change of representation can alter search difficulty dramatically even when theoremhood is unchanged.

3.7 Quotient and Invariant Engine

This module searches for equivalence relations, conserved quantities, symmetry classes, canonical forms, and compressed state spaces. It is especially useful when raw search contains many syntactically different states that are mathematically redundant.

3.8 Compass

Compass is a heuristic direction-finding system. It estimates which local moves or regions of theorem space appear promising. Compass may be learned, hand-designed, or hybrid. It never certifies correctness.

3.9 Scouts

Scouts are bounded exploratory workers. They receive limited budgets, limited permissions, and a local task. They may test candidate routes, search a neighborhood, attempt a lemma, compare representations, or estimate whether a branch is worth expanding.

The bounded nature of Scouts is important. A Scout is an exploratory service, not an unrestricted authority.

3.10 Player Zero

Player Zero is the controller. It schedules modules, allocates budgets, activates or deactivates Scouts, and decides which verified or speculative information should be routed where. It does not decide mathematical truth.

3.11 Quorum

Quorum aggregates multiple advisory judgments. It can be useful when several predictors, critics, or search estimators disagree. Quorum may influence scheduling but cannot override the Verifier.

3.12 Fuel

Fuel is the resource-accounting layer. It records and allocates computational budgets. A conforming implementation should charge expensive operations rather than treating them as free hidden assistance.

3.13 Partial Credit

Partial Credit estimates progress before final settlement. Examples include reduction in proof horizon, simplification of a target, successful elimination of branches, compression of a search space, or verified intermediate lemmas.

Partial Credit is useful for scheduling but must not be confused with final proof.

3.14 Sentinel

Sentinel detects stagnation, repetition, or suspicious operational patterns. It may trigger a change of representation, a resource reduction, a Scout shutdown, or escalation to a different search policy.

3.15 Adversary

Adversary is a critic. It attempts to break proposed arguments, identify hidden assumptions, construct counterexamples, detect invalid generalizations, and stress-test candidate conclusions.

3.16 Quarfill

Quarfill is a creative proposal generator. It deliberately searches outside the dominant route and produces unconventional conjectures, transformations, analogies, or decompositions. Its output is speculative by default.

3.17 Reviser

Reviser repairs candidate arguments or reformulates failed proposals in response to criticism, verifier rejection, or new information.

3.18 Firewall

Firewall governs operational permissions. It may restrict file access, network access, executable actions, model calls, external tools, or cross-module communication. Firewall is an operational safety mechanism, not a theorem checker.

3.19 Learner

Learner updates routing policies, heuristic models, ranking functions, representation choices, or resource-allocation policies from accumulated history. Learner must not rewrite BANK truth.

3.20 HISTORY

HISTORY records the sequence of actions, module calls, certificates, rejections, resource usage, policy changes, and important state transitions. It supports audit, reproducibility, and later learning.

4 What DATA MIND Can Do

The architecture is intended for problems in which a single proof search is too narrow. Its main capability is coordinated search across several mathematical and computational views of the same target.

A DATA MIND run can, in principle:

- search simultaneously for proof, refutation, reduction, model, or obstruction;
- retrieve verified mathematics relevant to the target;
- translate the target into alternative representations;

- identify symmetries and quotient redundant states;
- rank candidate next moves;
- allocate bounded Scouts to promising subproblems;
- preserve successful intermediate results in BANK;
- retain speculative but potentially useful ideas in FUTUREBANK;
- detect stagnation and change strategy;
- use critics to attack candidate proofs;
- use creative modules to propose nonstandard routes;
- learn which search patterns work across repeated tasks;
- produce an auditable history of the run.

The architecture is therefore particularly well suited to research programs where progress is incremental and where intermediate lemmas can materially change the future search landscape.

5 What DATA MIND Cannot Legitimately Do

The architecture does not make undecidable problems decidable by definition. It does not convert heuristic confidence into proof. It does not guarantee that a useful representation exists, that a learned predictor generalizes, or that additional Scouts will find a solution.

It also does not make computational cost disappear. If a search requires 10^{17} candidate examinations under a blind enumeration model, adding organizational structure does not by itself create information about the hidden target. Improvement requires exploitable structure, reusable verified information, a better representation, a better proposal distribution, or some other legitimate source of search advantage.

The architecture is therefore compatible with failure. A scientifically useful run may terminate with

- a proof,
- a refutation,
- a verified reduction,
- a smallest identified obstruction,
- a useful intermediate theorem,
- a negative search result within a declared budget,
- or an unresolved target with an auditable record of what was attempted.

This is preferable to silently converting an unsuccessful search into an unsupported claim.

6 Installation Philosophy

DATA MIND 3.0 is an architecture, not a single binary package. “Installing DATA MIND” therefore means constructing an implementation that satisfies the module interfaces and trust invariants.

A minimal installation can be surprisingly small. It needs:

1. a formal language or certificate format;
2. a Verifier;
3. BANK;
4. FUTUREBANK;
5. a Search Kernel;
6. a controller;
7. resource accounting;
8. HISTORY.

More advanced modules can then be added without changing the basic trust boundary.

7 A Minimal Installation Procedure

A practical installation can proceed in layers.

7.1 Step 1: Freeze the formal boundary

Declare the logic, proof format, axioms, trusted kernel, and certificate checker. Record versions and cryptographic hashes when appropriate.

The first installation artifact should answer:

> Exactly what must be true for the Verifier to accept a mathematical claim?

If that question is unclear, the installation is not yet ready.

7.2 Step 2: Create BANK and FUTUREBANK

Use separate stores. BANK entries require verifier acceptance. FUTUREBANK entries do not.

A simple BANK record might contain

$$(\text{id}, \text{statement}, \text{certificate}, \text{verifier version}, \text{timestamp}, \text{provenance}).$$

A FUTUREBANK record might contain

$$(\text{id}, \text{proposal}, \text{origin}, \text{estimated value}, \text{status}).$$

The two stores should never be merged merely for convenience.

7.3 Step 3: Install the Search Kernel

Connect one or more theorem provers, symbolic search procedures, SAT/SMT engines, proof enumerators, model finders, or domain-specific mathematical routines.

Every output must be typed. A module should return something like

`CandidateProof`, `CandidateCounterexample`, `CandidateReduction`, `Unknown`,

rather than an unstructured string that the rest of the system may misinterpret.

7.4 Step 4: Install Player Zero

Player Zero receives typed module outputs and allocates future work. The initial controller can be simple round-robin scheduling. More advanced learned scheduling can be added later.

A minimal scheduling loop is:

1. read the current target and verified state;
2. select a legal module call;
3. allocate Fuel;
4. execute the call;
5. send candidate certificates to the Verifier;
6. route accepted results to BANK;
7. route speculative results to FUTUREBANK;
8. append the event to HISTORY;
9. repeat until a stopping condition is met.

7.5 Step 5: Install operational containment

Firewall should explicitly declare what each module may read, write, execute, or contact. A theorem-search module normally does not need unrestricted system access.

7.6 Step 6: Add observability

Record module calls, runtime, verifier results, resource consumption, Scout births and deaths, controller decisions, and important state changes. Without observability, later learning and audit become unreliable.

7.7 Step 7: Run calibration problems

Before difficult research use, test the installation on targets with known outcomes. Calibration should include positive theorems, false conjectures, deliberately malformed certificates, expensive dead ends, and tasks where alternative representations are known to help.

8 A Reference Directory Layout

One possible implementation layout is

```
data_mind/
  verifier/
  bank/
  futurebank/
  search_kernel/
  retriever/
  representations/
  quotient_engine/
  compass/
  scouts/
  player_zero/
  quorum/
  fuel/
  partial_credit/
  sentinel/
  adversary/
  quarfill/
  reviser/
  firewall/
  learner/
  history/
  configs/
  tests/
```

The directory names are not mandatory. The separation of responsibilities is.

9 Module Contracts

Each module should have a machine-readable contract. A useful abstract form is

$$M = (I, O, P, C, V),$$

where

- I is the permitted input type,
- O is the permitted output type,
- P is the permission set,
- C is the cost-accounting rule,
- V is the version identifier.

For example, a Scout contract may permit reading the current target, selected BANK entries, and a bounded FUTUREBANK region; permit writing only candidate objects to a staging queue; and receive a fixed Fuel budget.

This makes upgrades easier because compatibility can be checked at the interface boundary.

10 Safe Upgrade

DATA MIND is designed to improve over time. The architecture therefore needs an upgrade protocol rather than ad hoc component replacement.

Suppose module M_v is replaced by M_{v+1}. A safe upgrade should check at least:

1. interface compatibility;
2. permission compatibility;
3. resource-accounting compatibility;
4. verifier independence;
5. regression performance;
6. failure behavior;
7. rollback availability;
8. HISTORY continuity.

A useful rule is

$$\text{upgrade permission} \not\Rightarrow \text{truth permission}.$$

A new Learner may become dramatically more accurate. A new Compass may improve proof search by orders of magnitude. A new Quarfill may generate much better conjectures. None of those upgrades should modify the acceptance rule of the Verifier unless the Verifier itself is undergoing a separately audited version change.

11 Verifier Upgrade

Verifier upgrades require special treatment because the Verifier defines mathematical trust.

If V_1 is replaced by V_2, the system should record whether

- V_2 accepts every certificate accepted by V_1;
- the underlying logic changed;
- the trusted kernel changed;
- certificate syntax changed;
- old BANK entries were rechecked;
- backward compatibility is guaranteed or merely expected.

A robust migration can preserve old acceptance provenance:

$$\text{BANK entry} = (T, \pi, V_1, \text{accepted}),$$

even if V_2 later also verifies π.

Historical truth claims should not lose their original verification context.

12 Learner Upgrade

Learner can be upgraded more aggressively because it has no direct truth authority.

A new learned model may be trained on HISTORY to predict

- which lemma is likely to be useful;
- which representation is likely to simplify a target;
- which Scout type is likely to pay for itself;
- which proof branch is likely to fail;
- which BANK entries are likely to transfer.

The correct evaluation question is not "Does the Learner know mathematics?" but rather

> Does the new Learner improve declared search metrics without violating interface, permission, or trust constraints?

Useful metrics include verifier-accepted results per unit Fuel, time to first useful lemma, reduction in redundant search, and calibration of predicted success probabilities.

13 Upgrading Compass, Scouts, and Quarfill

Compass upgrades change prioritization. Scout upgrades change local search behavior. Quarfill upgrades change proposal generation. These can all be powerful, but they should be evaluated separately.

A Compass benchmark might freeze a set of proof states and compare the rank assigned to eventually successful moves.

A Scout benchmark might compare

$$\frac{\text{verified useful outputs}}{\text{Fuel consumed}}.$$

A Quarfill benchmark might measure the fraction of proposals that later become verifier-accepted lemmas or lead to measurable search compression.

The system should resist the temptation to reward novelty alone. Creative output becomes valuable when it changes verified progress or useful search geometry.

14 How DATA MIND Is Used

A research run begins with a frozen target and a declared environment.

A useful run specification contains:

1. target T;

2. hypotheses H;
3. formal language;
4. Verifier version;
5. available BANK;
6. available modules;
7. Fuel budget;
8. Scout limits;
9. external-information policy;
10. stopping conditions.

Player Zero then coordinates search without changing these rules silently.

A typical cycle is

$$\text{retrieve} \to \text{represent} \to \text{propose} \to \text{criticize} \to \text{verify} \to \text{BANK/FUTUREBANK} \to \text{reallocate}.$$

The cycle need not be linear. Several branches may run concurrently.

15 Difficult Case I: A Large Theorem-Search Space

Consider a target theorem whose naive proof enumeration has an enormous branching factor. A monolithic prover may repeatedly explore equivalent or nearly equivalent states.

DATA MIND can attack the problem at several levels.

First, the Retriever searches BANK for reusable lemmas. Second, the Alternative-Math Presenter tries to expose a representation with smaller local branching. Third, the Quotient Engine searches for equivalence classes among proof states. Fourth, Compass ranks the remaining local actions. Fifth, bounded Scouts explore several high-value regions. Sixth, Adversary attacks candidate proof fragments before expensive continuation. Finally, accepted intermediate lemmas enter BANK and become reusable by all subsequent branches.

If the raw state space is S and an equivalence relation $\sim$ preserves the relevant continuation behavior, search may operate on

$$S/\!\sim$$

rather than S.

The important point is that the quotient is not assumed useful merely because it is smaller. Its usefulness is an empirical or formally analyzable property of the search process.

A successful run might not prove T immediately. It may instead verify a lemma L such that the remaining search horizon drops from

$$H(T) = 10^8 \quad \text{to} \quad H(T \mid L) = 10^5.$$

That is meaningful certified progress even before final settlement.

16 Difficult Case II: 3CNF SAT with Structural Locality

Let F be a 3CNF formula. A naive search can rebuild or rescan much of the formula after each assignment. A structurally aware representation records variable–clause incidence so that assigning variable x touches only clauses containing x or $\neg x$.

If every variable occurs in at most Δ clauses, one assignment can require only

$$O(\Delta)$$

local encoded-state updates rather than a full $\Theta(m)$ rebuild for m clauses.

DATA MIND can combine this with Compass and Scouts. Compass may rank variables or local simplifications. Scouts may explore bounded assignment neighborhoods. BANK may retain verified simplification lemmas or structural facts. Adversary may test whether a proposed pruning rule preserves satisfiability.

The Verifier boundary remains essential. A learned heuristic may say that a region is unlikely to contain a satisfying assignment, but pruning that region requires a sound certificate or a separately declared incomplete-search policy. Probability alone is not unsatisfiability.

This case illustrates the intended relationship between geometry and proof. Geometry may schedule attention. Structure may reduce update cost. Learned heuristics may concentrate search. Verification still determines which conclusions are mathematically trusted.

17 Difficult Case III: An Elusive Password-Like Target

Consider a search space of size N with exactly one hidden successful element and no observable structure correlated with the target.

If a search procedure examines b distinct candidates without replacement, the success probability is

$$\frac{b}{N}.$$

No amount of internal organizational complexity changes this if the observation interface reveals no target-correlated information.

DATA MIND is useful here partly because it can diagnose the limitation. Compass cannot legitimately create a gradient where none exists. Scouts cannot create information by multiplying. Player Zero cannot schedule its way into an oracle.

The correct system behavior is therefore to distinguish two cases:

1. *opaque search*: no exploitable structure is available, so performance is governed by coverage;
2. *structured search*: representations, constraints, partial certificates, or learned regularities create nonuniform value among actions.

This distinction prevents benchmark success on structured tasks from being misinterpreted as a universal ability to defeat blind enumeration.

18 Difficult Case IV: A Conjecture with Competing Proof and Refutation Routes

Suppose a conjecture C has resisted direct proof. A common failure mode is to spend all resources deepening one favored route.

DATA MIND can maintain competing settlements:

- Prover searches for a certificate of C;
- Refuter searches for a counterexample or certificate of $\neg C$;
- Retriever searches for related verified results;
- Alternative-Math Presenter changes representation;
- Quarfill proposes unconventional lemmas;
- Adversary attacks each candidate argument;
- Scouts explore bounded subcases.

Partial Credit can compare progress across these routes. For example, if a refutation branch reduces the possible counterexample domain from 10^{12} cases to 10^4 by verified reductions, Player Zero may allocate more Fuel there even if the initial prior favored proof.

The architecture therefore supports strategic flexibility without allowing strategic preference to become mathematical authority.

19 Difficult Case V: Scientific Hypothesis Generation

The same architecture can support scientific hypothesis generation if the evidence types are kept explicit.

Quarfill may generate hypotheses. Retriever may gather established results. Compass may rank experiments. Scouts may test bounded parameter regions. Adversary may search for confounders or alternative explanations. Learner may detect patterns across prior experiments.

But the final status labels must distinguish

$$\text{formal theorem}, \quad \text{empirically supported hypothesis}, \quad \text{speculative proposal}.$$

A verifier for a formal proof and an empirical validation pipeline are not the same object. DATA MIND should preserve that distinction rather than forcing all knowledge into a single confidence score.

20 Difficult Case VI: Recovery from a Misleading Learner

Suppose Learner becomes highly confident in a bad routing policy. It repeatedly directs Compass and Player Zero toward a family of dead ends.

A monolithic learned system may amplify the error. DATA MIND has several defenses.

Sentinel detects stagnation. Fuel limits the damage. Quorum can compare independent estimates. Adversary can challenge the dominant route. Quarfill can inject alternatives. HISTORY preserves the evidence needed to diagnose the failure. Most importantly, BANK remains intact because Learner cannot rewrite verified truth.

The recovery loop is

stagnation → Sentinel trigger → policy challenge → alternative routes → measured comparison → Learner revisio

This is one of the main reasons to separate learning from authority.

21 Resource Allocation and Scout Economics

Suppose each independent Scout has probability p of producing a useful result and each Scout costs c. If the value of at least one success is V, then the expected net value of m independent Scouts is

$$U(m) = V\left[1 - (1-p)^m\right] - cm.$$

The marginal value of Scout m is

$$Vp(1-p)^{m-1} - c.$$

A simple activation rule is therefore

$$Vp(1-p)^{m-1} > c.$$

This model is intentionally simple. Real Scouts are not independent and their outputs may overlap. Nevertheless, the formula illustrates an architectural principle: spawning workers should be treated as a resource decision, not as free parallel magic.

A mature implementation can estimate dependence, reuse, and information sharing explicitly.

22 BANK as a Search-Geometry Transformer

BANK is not only memory. A verified lemma can alter the geometry of future search.

Let $H_J(X)$ denote the remaining settlement horizon for target X under search regime J. If adding verified lemma L changes the horizon to $H_J(X \oplus L)$, define the BANK dividend

$$\mathrm{Div}_J(L; X) = H_J(X) - H_J(X \oplus L).$$

If acquiring L costs a, then the detour is strictly beneficial when

$$a < \mathrm{Div}_J(L; X).$$

This formalizes why an intermediate theorem can be valuable even when it does not change deductive closure in a deep logical sense. It may still shorten the operational path to settlement.

The distinction is important:

$$\text{same mathematical consequences} \not\Rightarrow \text{same search cost}.$$

DATA MIND is designed to exploit this gap while keeping the verified consequence relation fixed.

23 Learning from HISTORY

HISTORY permits the system to learn from more than final success or failure.

A run can generate training examples such as

- proof states paired with successful next moves;
- targets paired with useful representations;
- Scout configurations paired with realized dividends;
- verifier rejections paired with error classes;
- stagnation patterns paired with successful recovery actions;
- BANK lemmas paired with downstream horizon reduction.

This supports a richer learning objective than binary theorem solved / theorem unsolved.

For example, a routing model might estimate

$$\mathbb{E}[\mathrm{Div}_J(L; X) \mid \text{features of } L, X, J].$$

That prediction can guide search without becoming a theorem itself.

24 Operational Safety

Mathematical correctness and operational safety are different problems.

A perfectly sound theorem prover can still be embedded in unsafe software. Conversely, a tightly sandboxed program can still produce invalid mathematics.

DATA MIND therefore separates

$$\text{Verifier safety} \quad \text{from} \quad \text{Firewall safety}.$$

Verifier safety concerns whether accepted mathematical certificates are sound relative to the declared formal system. Firewall safety concerns what modules are permitted to do in the computational environment.

A production installation should apply least privilege. A Scout searching a combinatorial lemma should not automatically receive credentials, unrestricted network access, or permission to modify the Verifier.

25 Testing a DATA MIND Installation

Testing should occur at several levels.

25.1 Verifier tests

Include valid certificates, invalid certificates, malformed objects, boundary cases, and regression suites.

25.2 BANK tests

Ensure that unverified material cannot enter BANK and that accepted entries retain provenance.

25.3 Controller tests

Verify that Player Zero respects Fuel, Scout limits, permissions, and stopping rules.

25.4 Firewall tests

Attempt prohibited file, network, process, and cross-module operations.

25.5 Learning tests

Check that model updates change heuristics but cannot mutate BANK or Verifier rules.

25.6 Recovery tests

Deliberately install a poor Compass or misleading Learner and verify that Sentinel, Quorum, Fuel, and HISTORY make the failure observable and recoverable.

26 A Small End-to-End Example

Suppose the hypotheses are

$$P \to Q, \qquad Q \to R, \qquad R \to S,$$

and the target is

$$P \to S.$$

A simple DATA MIND run might proceed as follows.

Retriever identifies the chain structure. Quarfill proposes the intermediate lemma

$$P \to R.$$

The Search Kernel constructs a certificate from $P \to Q$ and $Q \to R$. The Verifier accepts it. The lemma enters BANK.

The new BANK state now contains a shorter route:

$$P \to R, \qquad R \to S,$$

from which the Search Kernel constructs a certificate of $P \to S$.

The example is elementary, but it illustrates the full architecture:

- proposal is not proof;
- certificate is checked;
- accepted intermediate structure becomes reusable;
- the search route becomes shorter;
- HISTORY can record the entire sequence.

The same pattern scales conceptually to much harder settings even when success is not guaranteed.

27 A More Difficult End-to-End Workflow

For a difficult target T, a mature run could look like this:

1. Freeze T, hypotheses, Verifier, Fuel, and external-information policy.
2. Retrieve relevant BANK material.
3. Generate several representations $R_1, \ldots, R_k$.
4. Estimate structural redundancy and candidate quotients.
5. Let Compass rank local moves in each representation.
6. Spawn bounded Scouts where estimated marginal value is positive.
7. Route promising candidate lemmas to Adversary before expensive proof search.
8. Send candidate certificates to the Verifier.
9. Add accepted results to BANK.
10. Recompute search horizons and partial-credit estimates.
11. Let Sentinel detect stagnating regions.
12. Use Quarfill and Reviser to generate alternatives.
13. Reallocate Fuel.
14. Stop on proof, refutation, declared obstruction, budget exhaustion, or another frozen stopping rule.

This is not a guarantee of discovery. It is a disciplined way to combine heterogeneous discovery mechanisms without losing track of what is actually known.

28 Why Modularity Matters

The architecture's main advantage is not merely that it contains many modules. It is that failures remain localizable.

If Compass performs poorly, replace Compass.

If Learner is miscalibrated, retrain Learner.

If Quarfill produces too much noise, tighten its budget or ranking threshold.

If a Scout family has negative marginal value, deactivate it.

If the Verifier rejects valid certificates because of an implementation bug, repair the Verifier under an explicit version transition and recheck affected BANK entries.

A monolithic system makes these distinctions difficult. DATA MIND treats them as first-class architectural boundaries.

29 Upgrade Without Epistemic Drift

The long-term danger in an adaptive research system is epistemic drift: a gradual loss of the distinction between what was proved, what was predicted, what was observed, and what was merely useful.

DATA MIND counters this by attaching status to objects rather than relying on memory or narrative.

A useful type discipline is

$$\texttt{Verified}(T), \quad \texttt{EmpiricallySupported}(H), \quad \texttt{Conjectured}(C), \quad \texttt{HeuristicScore}(x), \quad \texttt{OperationalEvent}(e).$$

Upgrades may improve transformations among these objects, but they should not silently coerce one type into another.

This type discipline is one of the simplest ways to make a growing mathematical AI system easier to audit.

30 Recommended Upgrade Sequence

A practical implementation should usually upgrade in the following order:

1. observability and HISTORY;
2. tests and reproducibility;
3. retrieval;
4. representation tools;
5. Compass;
6. Scouts;
7. Adversary and Reviser;
8. Quarfill;
9. Learner;
10. more sophisticated Player Zero policies.

Verifier changes should remain comparatively rare and heavily audited.

This sequence prioritizes the ability to measure improvement before introducing increasingly adaptive behavior.

31 Minimal Conformance Checklist

An implementation should not call itself conforming to DATA MIND 3.0 unless the following are explicit:

- the Verifier is identifiable;
- BANK and speculative memory are distinguishable;
- mathematical acceptance is not controlled by heuristic confidence;
- controller permissions are declared;
- expensive operations are charged to a resource budget;
- module outputs are typed or otherwise unambiguous;
- HISTORY records enough information for meaningful audit;
- upgrades are versioned;
- operational permissions are separable from mathematical authority;
- rollback or failure containment is possible for adaptive modules.

A system may implement only a subset of optional modules and still satisfy the architecture if the core trust boundary is preserved.

32 Research Questions

DATA MIND 3.0 suggests several concrete research directions.

First, how should partial credit be calibrated so that it predicts eventual verifier-accepted progress rather than merely local activity?

Second, when do learned geometric representations of theorem space transfer across domains?

Third, how should Scout dependence be modeled when workers share BANK and therefore cease to be independent?

Fourth, can one learn reliable estimates of BANK dividends before paying the full cost of proving a lemma?

Fifth, what stagnation signals distinguish a genuinely exhausted region from a region that merely requires a different representation?

Sixth, how should a controller balance proof, refutation, independence, and consistency-oriented search when their costs are highly asymmetric?

Seventh, what upgrade tests are sufficient to establish that a new Learner or Compass improves search without producing hidden distributional regressions?

These questions are empirical and mathematical. The architecture is intended to make them measurable.

33 Conclusion

DATA MIND 3.0 is a design for mathematical discovery under explicit epistemic control. It combines formal verification, persistent certified memory, speculative memory, search, retrieval, representation change, quotienting, heuristic guidance, bounded exploration, resource allocation, criticism, creative proposal generation, revision, safety containment, learning, and audit.

Its central claim is architectural rather than magical. Stronger intelligence is most useful when the system preserves a strict answer to the question: *why is this statement trusted?*

The intended answer is not "because the model was confident," "because many modules agreed," or "because the controller promoted it." The intended answer is that a declared Verifier accepted an appropriate certificate, or—for nonformal scientific claims—that the claim carries a separately declared empirical status.

That separation allows the rest of the machine to become adventurous. Compass may speculate. Scouts may explore. Quarfill may invent. Learner may adapt. Player Zero may reallocate the entire search. Adversary may attack everything. But BANK remains the record of what crossed the verification boundary.

The result is an architecture aimed not merely at finding more mathematics, but at finding it in a form that remains inspectable, upgradeable, and trustworthy as the surrounding machine becomes more capable.

References

[1] Brian Tenneson, *DATA MIND 3.0: Module and Interface Freeze*, October 2026.

[2] Brian Tenneson, *CDF: Certified Discovery Federation*, Version 12, October 2026.

[3] Brian Tenneson, *Password-Seeking with AMLD+0: Singleton Enumeration, SAT, and CNF Pruning*, September 2026.

[4] Brian Tenneson, *Exact Addressing, Proof Geometry Locality, and Verifier-Safe Hilbert Search*, Preliminary Release, October 2026.